法案系统建设说明书

文件版本：FINAL-V3.3	轻结构强执法闭环版
冻结目标版本：FROZEN-FINAL-V3.3
修订日期：2026-08-24
文件性质：治理系统源头建设规范 / Governance Bootstrap Specification
文件定位：规定法案正文、lawId、唯一 Owner、唯一 registry、任务范围、证据等级、渐进迁移和冻结发布
适用对象：Codex、AI Coding Agent、开发者、架构师、审计者、后续自动化执行者
交付形态：Markdown 为唯一权威源；PDF 为可选非权威派生物，仅在 GOVERNANCE-FREEZE.json 明确启用时生成并绑定
冻结状态：SPEC_FROZEN 状态只由当前有效 GOVERNANCE-FREEZE.json 对本文件字节的 SHA-256 绑定判定；本文件正文不自我声明当前冻结状态
冻结声明：本文件属于冻结产品，默认只读；只有人类明确授权修改本文件时才允许修改。
法条标识：全部 lawId 固定使用中性前缀 LAW-；任何历史项目名称、旧 fence 名称或旧 lawId 前缀不得进入本文件、法案正文、generated registry、证据或报告。

0. 本说明书的唯一身份

本说明书不是业务法案，不定义某个产品应该具备什么功能，也不替代项目的基础法案、功能法案、施工图、规格、契约或代码。

本说明书只拥有一个主权：

```
定义“法案系统本身必须如何建设、如何运行、如何防止 Codex 漂移、如何阻断第二链路”。
```

本说明书必须让一个从未接触项目的 Codex 在读完后能够直接回答并执行：

```
法案放在哪里；
法条按什么结构编写；
什么内容属于基础法案；
什么内容属于功能法案；
lawId 怎么编号和退役；
Owner 怎么唯一绑定；
唯一 registry 怎么生成和验真；
READ_ONLY、CHANGE、READY、RELEASE 如何分级；
CHANGE 在落笔前记录什么边界；
哪些范围扩张和迁移机制合法；
哪些越界、双重主权和伪 PASS 必须禁止；
质量系统怎么产出一份逐法条报告；
Exam 和 Dynamic Proof 在什么情况下必须存在；
什么证据足以宣布 PASS；
什么情况必须停止并返回 FAIL；
最终如何冻结，避免本文件被 Codex 自行修改。
```

本说明书不是可供 Codex“参考发挥”的原则集合。所有出现“固定”“必须”“只能”“不得”“精确”的条款均约束其明确点名的外部合同；Codex 不得改写这些法律效果。第 5 节、第 6 节第 9 项和第 68 节明确交给仓库的内部实现不因此被冻结。

第一部分 | 最高秩序与不可变公式

1. 法案系统的核心七件套

法案系统的核心固定由以下七类对象组成：

```
1. 冻结的《法案系统建设说明书》；
2. 一份《基础法案.md》；
3. 一份《功能法案.md》；
4. 由唯一 compiler 生成的 law registry；
5. 每个 capabilityDomain 唯一的 Owner Manifest；
6. 一套普通治理测试；
7. 对当前候选逐 lawId 产生裁决的一份质量报告。
```

唯一公式固定为：

```
法案系统
=
权威正文
+ 明确边界
+ 唯一 Owner
+ 完整索引
+ 任务范围控制
+ 可复放证据
+ 失败关闭执法
```

执法闭环固定为：

```
active lawId
-> 原子 enforcement binding
-> 稳定检查身份
-> 全部 required evidenceModes 精确覆盖
-> 非空可复放证据
-> 全分母无缺失、无重复、无额外项
```

文件可以共享，执法身份和法条裁决不得共享。轻量文件结构不得降低上述执法闭环。

以下组合均不构成法案系统：

```
只有文档，没有机器执法；
只有测试，没有 active lawId；
只有索引，没有权威正文；
只有 Owner 名称，没有路径主权；
只有正向目标，没有禁止面；
只有 PASS 汇总，没有逐法条报告；
只有 Codex 提示词，没有 active 法案；
只有目录和空壳，没有真实检查；
只有静态检查，却宣称真实运行已经成立。
```

2. 每条规则必须同时回答五个问题

每条 active lawId 必须精确回答：

```
要什么；
不要什么；
谁负责；
怎么证明；
违反后怎么办。
```

任何不能写出明确“不要什么”的要求，不得进入 active 法案。它只能继续讨论，不能取得治理主权。

2.1 边界闭合的唯一判定

“说清楚”固定等于同时写清正边界和负边界，不能只写目标，也不能只写禁令：

治理对象	要：必须做	不要：绝对禁区
法案	summary、唯一 Owner、required evidence、违法后果	forbidden[]、不得被下级覆盖
施工图	按需记录复杂拓扑、合同、状态和恢复	空壳、重复法案、第二 Owner/第二真相
规格	按需记录代码之外的字段、状态、错误和 contract identity	空模板、重复法案、隐式状态

治理对象	要：必须做	不要：绝对禁区
registry	忠实投影 active law、Owner、source 和按需合同	新增规范、人工维护、PASS/READY、第二索引
质检	正确证据类型、完整分母、一份逐法条报告、失败关闭	证据互相冒充、漏项放行、warning/partial/fake PASS
任务控制	READ_ONLY 零写入、CHANGE 轻记录、RELEASE 候选绑定	未声明写入、候选漂移、等级冒充和长期双重真相

边界判定规则固定：

1. “没有写不要”不等于默认禁止；它表示边界未闭合，必须 FAIL。
2. “没有写要”不等于执行者可以自行补全；它表示生产义务未定义，必须 FAIL。
3. 合法可选项必须进入精确枚举并同时写明选择者、前置条件和失败语义；不得把“可选”当作自由发挥区。
4. 同一对象在两个位置出现不同的必做、禁区、状态、字段、错误码或输出位置时，不得择一实现；必须先消除规范冲突。

3. 终态与建设阶段

active 法案描述需要长期成立的生产规则；施工可以分步进行。

阶段、MVP、preview、阶段性 feature flag 和迁移窗口可以合法存在，但必须标明：

```
当前完成范围；
未完成范围；
交付等级；
适用人群或流量；
进入下一等级的条件；
临时机制存在时的删除日期。
```

不得把阶段成果冒充完整生产终态，不得把未实现或未证明的能力记为 PASS。

4. 权威顺序与两类人类授权

必须把“人类授权”拆成两种不同主权，禁止混为一谈。

4.1 本次任务范围主权

人类对本次任务的原文只决定：

```
本次目标；
允许触碰的对象；
是否允许 law-amendment；
是否允许 source-spec-amendment；
是否为空白建制或既有治理迁移；
本次等级是 READ_ONLY、CHANGE、READY 还是 RELEASE；
是否涉及跨仓版本化合同。
```

任务原文不得直接降低现役法案、改变 Owner、跳过证据或授权第二链路。若普通任务原文与 active law 冲突，固定结果为 USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW。

4.2 规范内容主权

项目规范顺序固定为：

```
冻结的《法案系统建设说明书》
-> 《基础法案.md》
-> 《功能法案.md》
-> 按需系统或 capabilityDomain 施工图
-> 按需规格与机器契约
-> Owner Manifest
-> 唯一 law registry
-> 本次 CHANGE 范围记录或 RELEASE 候选绑定
-> 实现代码
-> 当前候选证据
-> 历史记录与讨论材料
```

解释规则固定为：

1. 人类明确授权“修订法案”只打开被点名法条及法定闭包的写权限；新内容仍必须符合本说明书。
2. 下级对象与上级冲突时，修改或删除下级对象；迁移期兼容机制只有满足第 43.2 节才合法。
3. 当前代码永远不能反向证明法案应该降低。
4. generated artifact 只投影，报告只证明，消费端副本只缓存；三者均不得取得源头主权。
5. 历史报告、旧提示词、工单、会议纪要和 Git 历史只用于追溯，不能取得当前主权。

第二部分 | 固定技术底座与目录

5. 实现技术由仓库决定

除第 68 节固定的五个公共治理命令身份外，本说明书不指定编程语言、运行时、依赖库、包管理器、测试框架或内部命令。每个仓库沿用与自身业务相容的技术栈，并只保留一个现役治理实现和一个 canonical quality 入口。

工具版本、依赖版本和锁文件由仓库自己的工程配置管理。更换实现技术不得建立第二治理主链，也不得改变本说明书规定的权威、Owner、范围、证据和交付边界。

projectChecks 由仓库按实际构建、类型、测试和部署需要定义，不取得 law verdict。CHANGE 只运行与本次差异相关的检查；READY 和 RELEASE 再运行各自要求的完整检查。

6. 最小固定路径

基础路径：

```
AGENTS.md

.github/instructions/agent-rules/
  基础法案.md
  功能法案.md

docs/governance/
  法案系统建设说明书-冻结定稿版.md
  GOVERNANCE-FREEZE.json

scripts/audit/
  domains/<capabilityDomain>/manifest.json
  generated/law-registry.generated.json
  reports/law-quality.report.json

.governance/runtime/
  change.json
  releases/<candidateId>/**

tests/governance/**
```

按需路径：

```
docs/blueprints/**
docs/specs/**
scripts/exam/**
scripts/dynamic-proof/**
```

规则固定：

1. READ_ONLY 不创建任何路径。
2. .governance/runtime/ 和 scripts/audit/reports/ 必须 gitignore，不得提交。
3. scripts/audit/generated/** 精确只有 law-registry.generated.json，只能由第 68 节固定的 registry 构建命令写入。
4. CI 报告写入仓库外 ephemeral path。
5. Manifest 登记 domain、路径主权和逐法执法 binding；不要求一法一文件。
6. Exam 与 Dynamic Proof 只在 active law 声明对应 evidenceModes 时建立；blueprint 与 spec 只在第 22、24 节条件成立时建立。
7. 不得以 v2/new/legacy/manual/compat/shadow 后缀建立长期同职路径；第 43.2 节迁移路径必须在删除日期前退出。
8. PDF 只有 freeze manifest 明确启用时才生成；PDF 不取得权威主权。
9. parser、compiler、quality 和测试的内部目录、文件名及实现技术由仓库决定，不属于本说明书冻结内容。

第三部分 | 冻结文件与两份 active 法案正文

7. 本说明书的冻结机制

docs/governance/GOVERNANCE-FREEZE.json 顶层只允许：

```
{
  "schemaVersion": 2,
  "status": "SPEC_FROZEN",
  "specPath": "docs/governance/法案系统建设说明书-冻结定稿版.md",
  "specSha256": "<64 lowercase hex>",
  "freezeVersion": "FROZEN-FINAL-V3.3",
  "amendmentAuthority": "explicit-human-only",
  "derivatives": {
    "pdf": {
      "enabled": false,
      "path": null,
      "sourceSha256": null,
      "fileSha256": null
    }
  }
}
```

冻结规则：

1. specSha256 对本说明书原始 UTF-8 字节计算 SHA-256；文件不得自写自身 hash。
2. SPEC_FROZEN 只锁定本说明书及 manifest 明确启用的派生物，不锁定 AGENTS、法案、registry 或实现，也不说明它们已经通过。
3. 第 68 节固定的治理入口在 CHANGE、READY、RELEASE 开始前核对说明书 freeze，并检查当前 AGENTS 是否符合第 45 节；READ_ONLY 直接读取并报告事实。
4. SYSTEM_PROVEN 与一个 candidateId 和 RELEASE context 绑定；任何 candidate 变化立即失效。
5. SYSTEM_PROVEN 不写回 freeze manifest，不反向取得规范主权；需要留档时只导出不可变 proof package。
6. PDF 默认关闭。开启时只作为 Markdown 的非权威派生物并绑定 source hash；PDF 不存在或未更新不得阻断默认关闭的冻结交付。
7. Markdown 标题、文件名或版本字符串不得单独宣称 SPEC_FROZEN；只有 freeze manifest 的当前有效 hash 可以证明冻结成立。

derivatives 只允许 pdf，pdf 只允许 enabled/path/sourceSha256/fileSha256，所有层级拒绝未知字段。enabled=false 时后三字段必须全部为 null；为 true 时三者必须非空，sourceSha256 必须等于 specSha256，fileSha256 绑定 path 指向的 PDF 原始字节。PDF 始终不取得权威主权。

8. active 法案正文精确为两份

人工维护并拥有 active law corpus 主权的文件精确为：

```
.github/instructions/agent-rules/基础法案.md
.github/instructions/agent-rules/功能法案.md
```

只有这两份文件中的 law-record fence 会被 active law parser 读取并进入 registry。

8.1 本说明书与 active law 的边界

本说明书规定法案系统的上位结构；第 66 节直接规定法定根治理与元治理规则。blank bootstrap 与 existing migration 必须按第 13、14、66 节把这些规则完整写入基础法案。

本说明书不保存 law-template fence，不作为运行时 active law source，不要求一法一 spec 文件、一法一 blueprint 文件、一法一 predicate 文件或一法一 mutant 文件。第 20、51 至 56 节规定的逐法逻辑执法不因文件共享而免除。

第 13 节只允许一个 law-record-example fence；它是结构示意，不进入 registry。

允许承接或投影法案、但不得创造新义务的对象只有：

```
受本说明书约束的 AGENTS.md；
按需 blueprint 与 spec；
Manifest；
唯一 law registry；
CHANGE 范围记录与 RELEASE 候选绑定；
一份逐 lawId 质量报告。
```

8.2 两份源头的闭环

防止第三法案不依赖全仓关键词扫描。active-law parser 只读取第 8 节固定的两份 active law；唯一 registry builder 只把这两份法案写入唯一 registry。其它 instruction、README、AGENTS、施工图、规格、注释、任务上下文和 generated artifact 均不能成为 active law source，也不得覆盖两份 active law。parser、builder 的内部文件名、实现语言和模块拆分不属于冻结内容。

本说明书的 law-record-example 只用于定义 parser 合同，不进入 active law corpus。CHANGE 范围记录与 RELEASE 候选绑定只记录机器字段，不取得法案主权。第 46 节只校验 AGENTS 与本说明书的符合性，不通过 freeze manifest 冻结 AGENTS 字节。

9. 两份法案的固定 front matter

《基础法案.md》开头精确使用：

```
---
documentType: base-law
schemaVersion: 2
status: active
authorityLevel: 100
containerOwner: law-system
lawIdPrefix: LAW
---
```

《功能法案.md》开头精确使用：

```
---
documentType: functional-law
schemaVersion: 2
status: active
authorityLevel: 90
containerOwner: law-system
lawIdPrefix: LAW
---
```

任何字段缺失、额外字段、值不一致、重复 key、重复 front matter 或第二元数据区均必须 FAIL。

10. 基础法案容纳长期稳定的根治理与元治理规则

基础法案的物理结构：

```
# 基础法案

## capabilityDomain: <stable-governance-domain>

### [LAW-<STABLE-GOVERNANCE-DOMAIN>-001-V1] ...
```

固定边界：

1. 基础法案只容纳不随普通业务迭代频繁变化、跨版本长期适用的治理 domain。
2. 第 66 节全部法定项目根规则与元治理 lawKey 必须完整出现。
3. 根规则覆盖权威、唯一真相与 Owner、任务范围、交付等级、迁移、证据、冻结和发布。
4. parser 对基础法案与功能法案使用相同算法。
5. 新增长期稳定治理规则必须对应不能由现役规则覆盖的独立风险，并获得人类明确授权。

基础法案不得包含产品流程、业务字段、页面、API、表、provider、业务状态机或单次任务范围。稳定治理法条可以约束 parser、compiler、质检和代码纯洁度的不变量，但不得把具体实现步骤、测试用例或报告格式写成法条。

11. 功能法案容纳随业务与能力持续迭代的法条

功能法案按 capabilityDomain 组织：

```
# 功能法案

## capabilityDomain: <domain>

### [LAW-<DOMAIN>-001-V1] <标题>
```

固定规则：

1. 第 66 节法定 law-system、audit-system lawKey 只属于基础法案；业务法条及随能力变化的治理法条只属于功能法案。
2. 业务 domain 与其它随能力变化的治理 domain 按 capabilityDomain 字典序排列。
3. 一个区块只容纳同名 domain 的 lawId。
4. 区块内按 lawKey 编号与版本升序排列。
5. 物理文件完整性 Owner 是 law-system；每个 block 的逻辑 Owner 是 metadata.capabilityDomain。
6. 修改一个 domain 不得格式化或整理其它区块。
7. parser、registry、scope、quality 的实现细节由普通测试约束，不得递归立法为固定法条工厂。

第四部分 | lawId 与法条原子语法

12. lawId 固定格式、lawKey 与历史谱系

lawId 正则固定为：

```
^LAW-[A-Z0-9]+(?:-[A-Z0-9]+)*-[0-9]{3}-V[1-9][0-9]*$
```

映射规则：

```
capabilityDomain = lower-kebab-case
lawId domain segment = capabilityDomain 转大写
lawKey = 删除末尾 -V<major> 的 lawId
```

示例：

```
capabilityDomain: account-person-workspace
lawKey: LAW-ACCOUNT-PERSON-WORKSPACE-001
lawId: LAW-ACCOUNT-PERSON-WORKSPACE-001-V1
```

编号与版本规则：

1. 每个 domain 从 001 单调递增。
2. V1 的 supersedesLawId 必须为 null。
3. Vn（n>1）必须精确 supersede 同 lawKey 的 V(n-1)，不得跳号。
4. requirement、forbidden、appliesTo、Owner、evidenceModes、failure条件或规范语义任一变化，major必须加一。
5. 同一 lawKey 只能有一个 active major；law-amendment 必须原子替换当前 major。
6. 第 43.2 节兼容窗口只适用于 spec 与实现，不允许旧、新 law major 同时进入 active law corpus；历史只在 Git 或 HISTORY_ONLY 路径。
7. 无关语义必须申请新编号，不得复用旧 lawKey。

版本关系只按当前 active law corpus 和 supersedesLawId 裁决。前代退役后不要求继续存在于 active corpus；parser 只验证 supersedesLawId 与当前 lawId 具有相同 lawKey，且 major 精确为当前 major 减一，不为此建立历史 registry。Git 历史只用于人工追溯，不进入普通 quality，也不得成为当前主权或 PASS 前提。

13. 每条 active 法条的精确 Markdown 结构

下面使用 law-record-example 只为避免本说明书自身被 active parser 误识别。active 两份法案中的实际 fence label 必须是 law-record：

````
### [LAW-<DOMAIN>-001-V1] <中文标题>

```law-record
{ ...canonical law JSON... }
```

active-law parser 必须把上述内层 `law-record` 视为被外层示例 fence 包裹的数据，不得把它计入 active
  block；active parser 只接受两份法案中未嵌套、未缩进、行内容精确等于三个反引号加 `law-record` 的 fence。

法条 section 不得增加背景故事、当前实现盘点、开发日志、TODO、方案比较或未进入 metadata 的隐藏要求。law-record
  只规定有期限迁移义务；具体期限、日期和删除验收进入关联 migration spec。实现落位进入按需 blueprint，机器合同进入按需
  spec，CHANGE 范围进入 runtime change record。

## 14. law-record JSON 精确字段

每个 block 只允许：

```json
{
  "schemaVersion": 2,
  "lawId": "LAW-<DOMAIN>-001-V1",
  "supersedesLawId": null,
  "status": "active",
  "tier": "P0",
  "capabilityDomain": "<lower-kebab-case>",
  "enforcementMode": "static",
  "evidenceModes": ["static.ast"],
  "summary": "必须……",
```
````

```
  "appliesTo": ["src/<domain>/**"],
  "excludes": [],
  "forbidden": ["不得……"],
  "dependsOnLawIds": []
}
```

字段约束：

14.1 schemaVersion、status、supersedesLawId

```
schemaVersion 固定整数 2；
status 固定 active；
V1 supersedesLawId=null；
Vn 精确指向同 lawKey 的 V(n-1)；前代已退役时无需保留其 active block，亦不得为验证版本关系建立历史 registry。
```

14.2 tier

只允许 P0、P1、P2；全部都是失败关闭，不存在 warning law。

14.3 enforcementMode 与 evidenceModes

enforcementMode 只允许 static、exam、dynamic、composite。

```
只有 static.* -> static
只有 exam.* -> exam
只有 dynamic.* -> dynamic
存在两类或三类 -> composite
```

evidenceModes 只允许：

```
static.filesystem
static.text
static.ast
static.types
static.contract
static.dependency
static.config
static.json
static.yaml
static.sql
static.migration
exam.golden
exam.property
exam.state-machine
dynamic.browser
dynamic.service
dynamic.database
dynamic.storage
dynamic.provider
dynamic.authorization
dynamic.recovery
dynamic.performance
```

数组非空、无重复、按上表排序。

每个 law-record 只允许一个因果一致的治理风险和一个最终裁决。义务不属于同一风险，或其 capabilityDomain、appliesTo/excludes、evidenceModes、tier 任一不同时，必须拆分 lawId；不得建立平行 clause registry。执法分母的唯一 key 是 lawId + bindingId + evidenceMode。

14.4 summary

```
以“必须”或“只能”开头；
只描述第 14.3 节的一个治理风险；
12 至 240 个 Unicode code point；
不得出现建议、尽量、可能、一般、视情况或原则上；
不得写开发日志、文件清单或单次任务范围；有期限迁移的机器字段进入第 69 节 migration contract，不塞入法条 summary。
```

14.5 appliesTo 与 excludes

```
仓库相对 POSIX glob；有效 UTF-8 NFC；不得绝对路径、..、反斜杠、负向 glob、brace、extglob、字符组或问号；
只允许精确文件、目录 /**、目录 /**/*.<literal-extension>；
数组字典序、无重复、无包含冗余；
appliesTo 非空；
只有 project-governance、law-system、audit-system 元法条允许根级 **/*；
excludes 只能排除 appliesTo 内明确子面，不得排除生产主链逃避执法。
```

14.6 forbidden

1 至 16 项；每项以“不得”或“禁止”开头；一项一个违法条件；不得以模糊阶段词豁免现役义务。有期限迁移的法条写明不得无期限并由关联 migration spec 提供第 43.2、69 节机器字段，不视为豁免。

14.7 dependsOnLawIds

只能引用当前 active lawId；按 lawId 排序且无重复；不得自引；依赖图无环；跨域依赖必须进入 registry，并在存在施工图时同时反映于相关施工图；不得分散 Owner。

15. 法条正文只写一次

law-record 是法条唯一正文。标题只提供阅读导航，不重复 summary、forbidden、evidenceModes 或违法后果。parser 验证标题 lawId、domain、版本关系和 JSON 字段，不维护第二份逐字镜像。

16. tier 违法后果与 parser 算法

tier 的违法后果固定为：

```
P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。
P1：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P1 finding；对应质量入口必须非零退出，并阻断合并和发布。
P2：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P2 finding；对应质量入口必须非零退出，并阻断合并。
```

16.1 law-parser 的精确解析算法

```
1. 读取原始字节；拒绝 BOM、非 UTF-8、非 NFC 路径和 CRLF；
2. 验证 front matter 从首字节开始，以精确 ---\n 打开和关闭；
3. 唯一 yaml parser 解析，禁止重复 key；
4. 逐行维护 Markdown fence 状态；
5. 只有精确行 ```law-record 打开法条 block；只有精确行 ``` 关闭；
6. 禁止未关闭、嵌套、缩进 fence 和 block 内注释；
7. block 必须是单一 JSON object，禁止未知字段；
8. 找到最近 level-3 law heading，一个 heading 精确一个 block；
9. 找到最近 level-2 capabilityDomain heading；基础法案和功能法案算法相同；
10. 验证标题 lawId、domain、major 与 supersedes；
11. 记录原始 lineStart/lineEnd、metadata hash、section hash；
12. EOF 后拒绝孤立 law heading 或 law-record；
13. 任一解析错误只产生结构冲突，不生成部分 registry。
```

16.2 其它 parser

Manifest、blueprint、spec 的 parser 必须拒绝重复 key、未知字段、空字符串、非法路径、未替换占位符和默认补值。其内部文件名、实现语言和模块拆分由仓库决定。

第五部分 | capabilityDomain、Owner 与 Manifest

17. capabilityDomain 命名与创建

capabilityDomain 正则固定为：

```
^[a-z0-9]+(?:-[a-z0-9]+)*$
```

每个 capabilityDomain 必须代表一个能够独立回答下列问题的完整责任域：

```
它拥有哪一种事实或能力；
它创建什么；
它读取什么；
它修改什么；
它裁决什么；
它明确不得接管什么。
```

不得创建以下 domain：

```
common、shared、utils、misc、core2、platform2、general、helpers、temporary、legacy、co
  mpat。
```

新 capabilityDomain 只有在用户明确授权新增能力域，并且现有 domain 无法合法承接时才允许创建。Codex 不得为了整理代码、追求优雅或减少文件行数自行新增 domain。

18. 物理容器 Owner 与逻辑 law Owner 分离

固定关系：

```
基础法案.md / 功能法案.md 的文件完整性 Owner = law-system
每个 law-record block 的逻辑 Owner = metadata.capabilityDomain
```

因此：

```
一个物理文件可以容纳多个逻辑 Owner；
一个 lawId 只能有一个逻辑 Owner；
Owner 不得通过拥有法案文件路径接管其它 lawId；
Audit 必须按 lawId block 定位主权，不得按 Markdown 文件整体绑定。
```

19. Manifest 固定路径

每个 active capabilityDomain 精确存在：

```
scripts/audit/domains/<capabilityDomain>/manifest.json
```

目录名与 manifest.capabilityDomain 必须相同。Manifest 只登记路径主权与第 20 节执法 binding，不得成为第二法案、独立测试计划或报告配置。

普通治理测试可以跨 domain 组织。含 static.* 法条的 domain 必须有可执行 inspector declaration；其文件名和模块拆分由仓库决定。

20. Manifest 登记 Owner、路径与原子执法 binding

每个 capabilityDomain 的 Manifest 顶层只允许：

```
{
  "schemaVersion": 4,
  "capabilityDomain": "<lower-kebab-case>",
  "status": "active",
  "lawIds": [],
  "lawBindings": {
    "LAW-...": {
      "staticBinding": {
        "bindingId": "<domain>.<law-key>.static",
        "checkId": "<domain>.<check-id>",
        "inspectorRef":
          "scripts/audit/domains/<domain>/inspector.ts#inspectLaw001",
        "targetSelectorId": "<domain>.<selector-id>",
        "evidenceModes": ["static.ast"],
        "negativeCases": [
          {
            "negativeCaseId": "<domain>.<negative-case-id>",
            "negativeCaseRef": "tests/governance/<file>#<case>",
```

```
          "expectedCode": "COMPONENT_FAILED"
        }
       ]
      },
      "examBindings": [],
      "proofBindings": []
    }
  },
  "ownedPaths": [],
  "observedPaths": []
}
```

examBindings[] 元素只允许 bindingId/examId/examRef/evidenceModes/negativeCases；proofBindings[] 元素只允许 bindingId/proofId/proofRef/evidenceModes/negativeCases。Ref 是仓库相对文件加稳定 export selector。三类 binding 的 evidenceModes 均非空、有序且无重复。

negativeCases[] 元素精确为 {negativeCaseId,negativeCaseRef,expectedCode}，三字段均非空。negativeCaseId 与 negativeCaseRef 全项目唯一；Ref 必须可执行；expectedCode 必须等于本负向条件应触发的确定性阻断码。

规则固定：

1. lawIds 精确等于该 domain 当前 active lawId。
2. lawBindings keys 与 lawIds 精确相等。
3. law 含 static.* 时 staticBinding 必须且只能有一个；不含时必须为 null。
4. staticBinding、examBindings、proofBindings 的 evidenceModes 分别精确覆盖 law 的 static.*、exam.*、dynamic.*；每个 mode 恰好出现一次。
5. bindingId、checkId、examId、proofId、inspectorRef、examRef 和 proofRef 全项目唯一。全部 Ref 必须精确存在；每个 binding 的 negativeCases 非空；staticBinding 的 targetSelectorId 非空。
6. 一次 check 只裁决一个 lawId。不得把 domain 的全部 lawIds 批量绑定给一个汇总结果。
7. inspector、predicate、Exam和Proof可以共享模块或辅助代码；不得共享lawId、bindingId、执行结果或证据分母。
8. ownedPaths 表示唯一 primary 写主权；observedPaths 只读。路径只允许精确文件或目录 /**；Owner 重叠必须 FAIL。
9. lawIds 按 lawId，binding 数组按 bindingId，路径数组按 UTF-8 bytes 升序；全部无重复。
10. Manifest 不得写法条正文、PASS 条件、豁免、report sink 或新义务。

21. 候选文件角色与 ownership

候选路径只分为：

```
GOVERNANCE_AUTHORITY；
GENERATED_REGISTRY；
QUALITY_REPORT；
RUNTIME_IGNORED；
TEST_ONLY；
HISTORY_ONLY；
PRODUCTION。
```

规则固定：

1. 本说明书、freeze、AGENTS、两份 active law、Manifest 及按需 blueprint/spec 为 GOVERNANCE_AUTHORITY。
2. 第 26 节唯一 registry 为 GENERATED_REGISTRY；第 57 节报告为 QUALITY_REPORT；.governance/runtime/** 为 RUNTIME_IGNORED。
3. tests/ 及未被生产消费的 fixture/example 为 TEST_ONLY；明确 docs/history/ 为 HISTORY_ONLY；其余候选路径默认为 PRODUCTION。
4. PRESENT 的 PRODUCTION 文件必须命中一个且仅一个 Manifest ownedPaths。
5. 非生产路径被生产 import、发布包、配置或状态写入消费时，按 PRODUCTION 裁决。

6. DELETED 不读取工作树字节；删除权限按 baseSha、allowedDeletePaths 和原 Owner 验证。
7. rename 视为旧 path DELETE 与新 path PRESENT。
8. symlink 不得越出仓库根；submodule、unmerged index 和特殊设备文件必须阻断。

固定结果：

```
零 Owner -> UNOWNED_PRODUCTION_FILE；
多 Owner -> MULTIPLE_PRIMARY_OWNERS；
非生产角色被生产消费 -> ROLE_ESCALATION_REQUIRED；
未授权删除 -> UNAUTHORIZED_PRODUCTION_DELETE。
```

22. 施工图按复杂度建立

SYSTEM-BLUEPRINT.md 与 capabilityDomain blueprint 不再是每个仓、每个 domain、每条法案的强制配件。

出现以下任一情形时，才必须建立机器可读施工图：

```
三个以上 domain 的依赖拓扑；
跨域合同；
跨仓合同；
多状态恢复流程；
无法从代码与 spec 直接确定的 Owner 或调用方向。
```

施工图的最小机器 block：

```
{
  "schemaVersion": 1,
  "blueprintId": "<stable-id>",
  "status": "active",
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "contractRefs": [],
  "dependencyEdges": [
    {
      "from": {"repo": "local", "domain": "<capabilityDomain>"},
      "to": {"repo": "local", "domain": "<capabilityDomain>"},
      "contractRef": null
    }
  ],
  "prohibitedDependencyEdges": [
    {
      "from": {"repo": "local", "domain": "<capabilityDomain>"},
      "to": {"repo": "local", "domain": "<capabilityDomain>"},
      "lawId": "LAW-..."
    }
  ]
}
```

每个 blueprint 的 lawIds 与 contractRefs 精确定义该 blueprint 的 law-to-contract 关系；registry 同时投影第 24 节 active specs 的显式 lawIds。不得依赖 Markdown 标题、行号或自然语言推断关系。

lawIds 按 lawId、contractRefs 按路径的 UTF-8 bytes 升序排列，均无重复。

edge endpoint 精确为 {repo, domain}。repo 只允许字面值 local 或 active cross-repo contract 中的 canonical owner/repo；domain 必须是对应仓的现役 capabilityDomain。from 表示依赖方或调用方，to 表示被依赖方或提供方；两端不得相同。

dependencyEdges[] item 只允许 from、to、contractRef。contractRef 只允许 null 或 top-level contractRefs 中的 active spec 路径；跨域或跨仓合同存在时不得为 null。

prohibitedDependencyEdges[] item 只允许 from、to、lawId。parser 只验证 lawId 同时存在于 top-level lawIds 且为 active；该 law 是否禁止此方向由普通治理测试按明确合同和依赖关系裁决，不要求 parser 解释自然语言。施工图不得自行创造禁令。

两个数组分别按 from.repo、from.domain、to.repo、to.domain、最后一个字段的 UTF-8 bytes 升序排列。同一数组不得重复；相同 from/to 不得同时出现在允许与禁止数组；未知字段、非法 endpoint、悬空引用、逆序、重复或允许/禁止冲突必须 FAIL。

blueprint 可以是独立 JSON 文件，也可以是 Markdown 中唯一的 blueprint-record JSON fence；builder 只读取该机器 block，不解析正文形成隐含合同。

施工图只记录 topology、Owner、contract 和 prohibited edge；不得为每个 lawId 建立形式化 anchor。

简单 domain 可以只使用法案、Manifest、代码与测试，不得为了满足目录形式创建空 blueprint。

23. 施工图正文

施工图只写当前系统需要长期维持的结构，不设固定十五章，不要求空章节。

正文应覆盖实际存在的：

```
终态对象；
Owner；
输入输出；
持久化与状态；
调用方向；
错误恢复；
安全边界；
验收条件。
```

不存在的对象不得以占位章节、TODO 或未来规划写入。

24. 规格按机器合同建立

只有法条涉及字段、API、事件、存储、状态机、配置、安全合同或跨仓合同，且代码类型本身不足以成为唯一机器真相时，才建立 spec。

spec 必须具有：

```
specId；
version；
Owner；
适用 lawIds；
精确合同内容。
```

spec 的最小 schema：

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<stable-id>",
  "status": "active",
  "version": 1,
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "specKind": "typed-contract",
  "contract": {}
}
```

lawIds 按 lawId 排序且无重复。

specKind 只允许 typed-contract、api、event、storage、state-machine、config、security、migration、cross-repo-contract、contract-pin。specKind 唯一决定 contract 使用的 schema；同一 specKind 和 schemaVersion 不得同时对应两套 schema，也不得由文件路径、Owner 或运行参数改选 schema。spec parser 校验公共外壳后，按 specKind 进入唯一合同 schema；schema 的内部实现位置和技术由仓库决定。公共外壳未知字段、缺失 Owner、空 lawIds、未知 specKind、合同不符合对应 schema 或未被 active law 引用必须 FAIL。

version 必须是正整数；contract 的规范内容发生变化时递增，旧版本只可在第 43.2 节兼容窗口内继续存在。spec 不得重复法条正文，不得为每条 lawId 强制建立一份文件，不得用空数组和模板字段制造形式完整。

跨仓合同使用 provider descriptor 与 consumer pin。破坏性改变提升 version；消费者在声明的兼容窗口内独立部署，不要求同一任务同步完成全部仓库 RELEASE。

25. 法案、Manifest、施工图、规格与任务记录的分工

法案：规定必须、禁止、Owner、证据等级与违法后果；
Manifest：登记 domain、lawIds、路径主权与原子执法 binding；
施工图：按需记录复杂拓扑和调用边界；
spec：按需记录代码之外的机器合同；
change record：记录本次允许写入、删除的路径和 baseSha；
registry：投影当前 active laws、Owner、合同引用与 enforcement binding；
质量报告：逐 lawId 记录当前候选的裁决与证据。

任何对象不得重复取得另一对象的主权。

第六部分 | 唯一 law registry 与 canonical 生成

26. 唯一 registry

唯一机器索引固定为：

```
scripts/audit/generated/law-registry.generated.json
```

不得建立平行 law index、authority census、enforcement-bindings 文件、audit-plan 文件、inspector-registry 文件或第二 generated projection。第 20 节的 enforcement binding 必须嵌入本 registry 的对应 law record。

registry 只投影当前 active law、Owner、source location、evidenceModes、contract refs 和 enforcement binding；不拥有规范主权或 PASS 权。

27. registry 顶层 schema

```
{
  "schemaVersion": 4,
  "sourceSpec": {"path": "...", "sha256": "..."},
  "lawCorpus": {
    "baseLawPath": "...",
    "baseLawSha256": "...",
    "functionalLawPath": "...",
    "functionalLawSha256": "..."
  },
  "laws": [],
  "domains": []
}
```

domains[] 每项只允许：

```
{
  "capabilityDomain": "...",
  "manifestPath": "...",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

全部数组按稳定 key 排序；禁止时间、机器名、用户名、绝对路径和随机值。

28. registry law record

```
{
  "schemaVersion": 2,
  "lawId": "LAW-...",
  "status": "active",
  "supersedesLawId": null,
  "tier": "P0",
  "capabilityDomain": "...",
  "enforcementMode": "static",
  "summary": "...",
  "appliesTo": [],
  "excludes": [],
  "forbidden": [],
  "evidenceModes": [],
  "dependsOnLawIds": [],
  "source": {
    "path": "...",
    "lineStart": 1,
    "lineEnd": 10,
    "sectionSha256": "..."
  },
  "contractRefs": [],
  "enforcement": {
    "staticBinding": null,
    "examBindings": [],
    "proofBindings": []
  }
}
```

contractRefs 只收录 lawIds 包含该法条的 active blueprint/spec，并精确使用：

```
{"path": "repo/relative/path", "sha256": "<64 lowercase hex>"}
```

数组按 path 的 UTF-8 bytes 升序排列，sha256 绑定对应 artifact 原始字节；没有按需合同的法条必须为空数组。enforcement 精确投影第 20 节对应 binding，不收录运行结果或 report sink。

29. hash

文件 hash 使用原始 UTF-8 字节；law section hash 使用法条完整原始字节；registry hash 使用第 30 节 canonical JSON。

Git object id 使用当前仓库 object format 的小写十六进制，不固定为 40 位；同一候选中的 Git id 必须使用同一 object format。

不得把时间、路径之外的机器环境、报告或运行结果写入 source hash。

30. canonical JSON

对象 key 按 Unicode code point 升序递归排序；数组保持其所属 schema 规定的顺序。registry 的 laws 按 lawId、domains 按 capabilityDomain、contractRefs 按 path 排序；字符串数组按各自字段规则排序。缩进 2；单 LF 结尾；禁止注释、尾空格、NaN、Infinity、undefined 和时间字段。

31. 唯一 registry builder

仓库只允许一个承担 registry 生成主权的 builder，并由第 68 节固定的 audit:build 调用。builder 的内部文件名、实现语言和模块拆分由仓库决定。

输入只包括：

```
说明书与 freeze manifest；
当前 AGENTS；
两份 active 法案；
全部 active Manifests；
全部 binding 指向的 inspector、Exam、Proof 和 negative-case declarations；
固定按需目录中全部 status=active 的 blueprints/specs。
```

唯一输出是 law-registry.generated.json 的 canonical bytes。builder 必须验证 law、binding、declaration 与 evidenceMode 精确对等；不读取生产候选、历史报告、网络、数据库、浏览器、墙钟或随机源，不输出 verdict。

32. source-integrity preflight

builder、projection check 与 READY、RELEASE quality 共用以下只读检查：

```
说明书 freeze；
AGENTS 是否符合第 45 节；
两份 active law 结构；
active law source 精确为固定两份；
Manifest Owner 与路径重叠；
active law、Manifest binding、declaration 和 registry 的全分母对等；
按需 blueprint/spec 的 schema、lawIds 与 Owner 完整性；
path encoding 与 case collision。
```

CHANGE 只验证说明书 freeze、AGENTS 边界及本次实际触碰的治理 source；不遍历全量 registry、ownership 或未受影响合同。READY、RELEASE 执行上述完整 preflight。freeze 不一致时不得宣称 SPEC_FROZEN。

失败只输出确定性 code，不写 registry 或报告。

33. registry 构建

audit:build 在内存生成唯一 registry，验证可重新解析后以临时文件和原子 rename 替换目标文件。失败时保留旧文件，不得留下半写状态。

audit:build 不写质量报告，不自动补法案、Manifest、blueprint 或 spec。

34. projection check

audit:projection:check 重新调用同一 builder，比对唯一 registry exact bytes，并证明检查前后候选未变化。

registry 缺失返回 GENERATED_FILESET_INVALID；字节陈旧或被手改返回 GENERATED_PROJECTION_STALE。检查不得自动调用 audit:build 修复。

第七部分 | 任务范围与交付等级

35. 任务按交付等级分级

任务只分为 READ_ONLY、CHANGE、READY、RELEASE 四级。不得以较低等级冒充较高等级。

分类顺序固定：人类明确禁止写入时为 READ_ONLY；明确要求生产发布、最终发布证明或 SYSTEM_PROVEN 时为 RELEASE；明确要求合并、部署前就绪或 READY 验收时为 READY；其它包含仓库写入的任务为 CHANGE。执行者不得自行降级；人类未要求更高等级时不得自行升级。

35.1 READ_ONLY

READ_ONLY 只读取权威与目标文件并回答；不加锁、不建立事务或 artifact、不写入 .governance/runtime/**。

只读结论不得宣称代码已经修改、候选已经验证或系统已经发布。

35.2 CHANGE

CHANGE 允许在明确路径内修改。首个目标写入前，先建立第 36 节唯一 change record，再运行只读 governance:entry；不加锁，不生成其它任务事务 artifact。

CHANGE 只证明本次差异未越界并通过与差异相交的普通测试；不得据此宣称 READY、RELEASE 或 SYSTEM_PROVEN。

35.3 READY

READY 表示当前候选已通过全部 projectChecks、全部 active Static、全部 active Exam、required mutants 和完整逐法报告；声明的 dynamic.* 尚未取得。

READY 不得替代真实生产环境、外部依赖或动态运行证据。

35.4 RELEASE

RELEASE 是唯一执行完整候选绑定、全部 active bindings、完整 mutant catalog、全量逐法报告和发布证明包的等级。

只有 RELEASE 可以产生 SYSTEM_PROVEN。

36. CHANGE 的唯一范围记录

CHANGE 首次目标写入前只允许建立一份 gitignored record：

```
{
  "baseSha": "<git object id>",
  "allowedWritePaths": [],
  "allowedDeletePaths": []
}
```

规则固定：

1. baseSha 是开始修改时的 Git HEAD。
2. 两个路径数组只能包含仓库相对 POSIX 精确路径或目录 /**，无重复、无重叠。
3. record 只允许三个法定字段，不得保存请求副本、任务摘要、闭包、路由、上下文、回执或证据。
4. record 不构成规范主权，不得提交。
5. 修改范围扩大时，必须在首次触碰新增路径前更新同一 record 并重新运行只读 entry；不得创建第二份 record。
6. 首次目标写入前必须检查目标路径的 Git 状态；目标已有用户改动且人类未明确授权在其上继续编辑时，立即停止。三字段 record 只证明路径范围，不得声称能够证明改动作者或自动恢复用户字节。

37. CHANGE 的范围判定

允许修改的路径仅来自：

```
人类明确点名的路径；
完成该修改不可分割的同一合同路径；
由唯一生成器确定发生变化的派生路径。
```

第二、三项只有在任务确实需要时才加入；不得预授权全仓，不得以整理、抽象、未来复用或顺手修复扩大范围。

READ_ONLY 不执行本节。

38. 范围扩张

范围扩张只允许唯一生成器派生物、同一 Owner、同一合同、active law 与当前等级直接要求的证据，或旧路径退休的直接闭包。

扩张前必须把新增路径写入同一 change record。record 仍只保存 baseSha、allowedWritePaths 和 allowedDeletePaths，不建立其它范围事务对象。

新增 API、CLI、事件、队列、表、列、环境变量、依赖、监听器、外连主机、浏览器权限或跨仓合同，仍须有人类明确授权或现役法案的直接依据。

39. 法案与既有治理迁移

修改本说明书必须有人类明确授权 source-spec-amendment；授权只覆盖本说明书、freeze manifest 和已启用派生物，不自动授权修改 AGENTS、active law、registry 或实现。正文定稿后才计算并写入新的 specSha256。

39.1 法案修订

修改 active 法案必须有人类明确授权 law-amendment。

修订可以在分支、工作树或隔离目录中验证；不得要求所有关联实现必须在同一任务达到 RELEASE。修订后的 active law、registry 和实现只有在各自完成时才取得对应状态，未完成项必须如实标记；law-amendment 不改变本说明书的 freeze。

39.2 既有治理迁移

既有治理允许分步迁移。每一步必须声明当前权威、目标权威、受影响路径、验证方式和剩余工作；不得把中间状态冒充 READY 或 RELEASE。

迁移期间允许第 43.2 节规定的有期限兼容机制。

40. RELEASE 候选绑定

进入 RELEASE 时必须固定：

```
candidateId；
Git commit；
法案与 registry hash；
唯一逐法质量报告 hash；
全部 required Dynamic Proof hash；
跨仓合同版本与参与仓发布结果。
```

完整绑定由 quality:check 的 RELEASE 分支在最终报告通过后写入 .governance/runtime/releases/<candidateId>/**。第 B.9 节的 proof package 就是唯一 RELEASE 绑定 artifact，不得另建 release receipt 或同义绑定层。绑定缺失或 candidate 漂移时，RELEASE 失败。CHANGE 与 READY 不生成该完整绑定。

41. 固定阻断码

任务控制的跨实现稳定阻断码精确为：

```
FREEZE_HASH_MISMATCH
ENTRY_CONTRACT_MISMATCH
USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW
CHANGE_RECORD_MISSING
CHANGE_RECORD_INVALID
BASE_SHA_MISMATCH
SCOPE_DIFF_OUTSIDE_ALLOWED_PATHS
DIRTY_TARGET_NOT_AUTHORIZED
LAW_AMENDMENT_REQUIRED
```

```
SOURCE_SPEC_AMENDMENT_REQUIRED
RELEASE_CANDIDATE_UNBOUND
RELEASE_EVIDENCE_MISSING
CANDIDATE_DRIFT
EXTERNAL_SURFACE_AUTHORITY_MISSING
```

执法闭环至少使用以下确定性阻断码：

```
LAW_BINDING_MISSING
LAW_BINDING_DUPLICATE
EVIDENCE_MODE_UNBOUND
UNDECLARED_EVIDENCE_BINDING
INSPECTOR_MISSING
EXECUTOR_REF_MISSING
NEGATIVE_CASE_MISSING
NEGATIVE_CASE_REF_MISSING
PREDICATE_IDENTITY_MISMATCH
TARGET_DENOMINATOR_MISSING
TARGET_OUTSIDE_DOMAIN_AUTHORITY
EMPTY_ASSERTIONS
EMPTY_EVIDENCE_REFS
DUPLICATE_EVIDENCE_COVERAGE
ACTIVE_LAW_DENOMINATOR_MISMATCH
COMPONENT_FAILED
MUTANT_NOT_DETECTED
```

READ_ONLY 只在目标或权威不可读取时停止；不得因为缺少 runtime task artifact 而停止。

42. Codex 范围禁区

未经明确授权，Codex 不得：

```
修改 change record 之外的路径；
新增产品范围、公开接口、持久化结构或外部权限；
建立第二法案、第二 registry、第二 Owner、第二 compiler 或第二质量入口；
用 fixture、mock、no-op 或固定返回冒充生产能力；
用 CHANGE 或 READY 结论冒充 RELEASE；
覆盖、删除、恢复或移动用户原有改动。
```

迁移桥、用于迁移的 feature flag、dual-read、dual-write 和 shadow traffic 依第 43.2 节裁决，不作绝对禁止。长期产品 feature flag 由对应业务 law 管理，不得用于暗中保留旧真相源。

43. 交付与迁移

43.1 渐进建设

渐进建设精确使用第 3 节规则，不新增状态或豁免。

43.2 有期限兼容机制

兼容桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic 或临时 adapter 只有同时满足以下条件才合法：

```
唯一 Owner；
旧、新路径身份明确；
开始日期；
监控指标；
回滚条件；
最长存续期限；
删除日期；
删除验收标准。
```

兼容机制不得形成两个长期真相源；到期未删除必须 FAIL。

43.3 跨仓交付

跨仓通过版本化合同独立部署。合同内容变化递增 version；破坏性改变同时声明兼容窗口和消费者迁移期限。

一仓 CHANGE 或 READY 不得冒充另一仓完成；只有整体 RELEASE 声明才要求全部必要参与仓的发布结果。

44. 最终 scope check

CHANGE、READY、RELEASE 均以 baseSha 到当前候选的实际 Git 差异为分母。

governance:scope:check 必须验证：

```
实际 WRITE 全部属于 allowedWritePaths；
实际 DELETE 全部属于 allowedDeletePaths；
首次写入前的 dirty target 已获得明确授权，否则未被触碰；
法案修改具有对应人类授权；
generated bytes 与唯一生成器对同一输入的输出完全一致；
新增生产路径具有唯一 Owner；
新增外部 surface 具有现役依据。
```

任一不成立，返回第 41 节对应阻断码并非零退出。

45. AGENTS.md 的边界

AGENTS 只写入口、权威顺序、READ_ONLY 零写入、CHANGE 范围记录、交付等级、RELEASE 候选绑定、有期限迁移、active law 修改授权和失败关闭；不得复制业务法案、手工路由表或完整质量流程。

46. 治理入口

READ_ONLY 可以直接读取，不强制运行 entry。

CHANGE 建立第 36 节 record 后、首次目标写入前运行 governance:entry。entry 从该 record 取得 baseSha 与目标路径，检查说明书 freeze、AGENTS 边界、唯一质量入口和目标路径初始 Git 状态。READY、RELEASE 额外检查法案与 registry。未获授权的 dirty target 返回 DIRTY_TARGET_NOT_AUTHORIZED。entry 只读，不创建 task artifact，不输出 PASS 文件。

47. 读取规则

```
READ_ONLY：读取回答问题所需的权威和目标文件，不落盘；
CHANGE：读取相关法案、registry 和目标文件；
READY：在 CHANGE 基础上读取全部 active law、Manifest、blueprint、spec 与验收合同；
RELEASE：读取完整候选、全部 active laws、registry 和发布证据。
```

说明书 freeze 不匹配或 AGENTS 违反第 45 节时，CHANGE、READY、RELEASE 停止；READ_ONLY 仍可读取并报告事实。

第八部分 | 质量与证据

48. Static Audit 的边界

Static Audit 只证明当前候选的文件、文本、AST、类型、schema、合同、依赖、配置和迁移声明。

Static Audit 不得启动外部系统，不得把测试夹具、历史报告或截图冒充当前运行证据，不得输出 RELEASE verdict。

49. CandidateIdentity

CHANGE 与 READY 以 baseSha 到当前工作树的实际 Git 差异为候选分母。

READY 的 candidateFingerprint 对 {baseSha,diffs,baseLawSha256,functionalLawSha256,registrySha256} 的 canonical bytes 计算 SHA-256。diffs[] 元素只允许 {status,path,sha256}，status 只允许 A/M/D，D 的 sha256 为 null；按 path/status 排序。未跟踪文件必须以 A 进入。

RELEASE 只允许绑定一个已提交的 Git tree；受治理的生产文件不得存在未提交或未跟踪变化。candidateId 依第 30 节对以下对象的 canonical JSON bytes 计算 SHA-256：

```
{
  "gitCommit": "<git object id>",
  "baseLawSha256": "<64 lowercase hex>",
  "functionalLawSha256": "<64 lowercase hex>",
  "registrySha256": "<64 lowercase hex>"
}
```

Git commit 已绑定该提交的完整 tracked tree，禁止再维护第二份 path 清单。Dynamic Proof、质量报告和 proof package 分别以第 B.9 节 hash 绑定，不进入 candidateId。绝对路径、时间、机器名、用户名和报告内容不得进入 candidateFingerprint 或 candidateId。

50. candidate drift

质量检查开始后，候选发生变化时：

```
当前检查失败；
本次临时结果作废；
不得发布质量报告；
RELEASE 必须重新绑定 candidateId。
```

CHANGE 不生成 candidate receipt。READY 报告必须绑定 candidateFingerprint；RELEASE 报告必须绑定 candidateId。

51. 一套普通治理测试

治理测试可以按 domain 或技术类型拆分文件，由仓库现有测试框架组织。只有 canonical quality 入口可以汇总治理结论；测试文件数量和目录结构不构成第二质量入口。

测试至少覆盖：

```
法案 parser；
lawId 与当前版本关系；
active law source allowlist；
Manifest Owner；
registry deterministic bytes；
scope；
证据分层；
禁止伪 PASS。
```

测试、inspector 和辅助代码可以共享，但法条映射只来自第 20 节 binding。一次 predicate 执行只返回一个 lawId 和一个 bindingId；共享运行必须拆分为各法独立结果。普通测试不得自行绑定 domain 全部 lawIds。

52. 静态证据分母

静态证据分母精确为：当前候选中命中 active law appliesTo 的路径，减去命中 excludes 的路径，再加入该 lawId 全部 active contractRefs。每个路径必须命中该 domain Manifest 的 ownedPaths 或 observedPaths，否则 FAIL。targetSelectorId 只标识该法条的这次确定性查询，不得选择其它算法、覆盖这一分母或在运行时裁剪目标。

每个 staticBinding 必须在执行前使用 targetSelectorId 产生 {queryRef,count,sha256} target set；queryRef 必须是第 55 节 scope evidenceRef，绑定 lawId、targetSelectorId、选择输入与规则。目标仓库相对 POSIX paths 按 UTF-8 bytes 排序，每条写为 path + LF，空集合为零字节；sha256 绑定该原始字节，count 等于路径数。后续 assertion 只能消费该 set。纯 absence 检查可以 count=0，但 queryRef 和 sha256 必须非空。

discovery、read、decode 或 parse 失败必须 FAIL。

以下路径不得作为业务实现已经成立的充分证据：

```
tests/**
fixtures/**
examples/**
artifacts/**
coverage/**
docs/history/**
scripts/audit/reports/**
```

53. Inspector 执法合同

inspectorRef 指向该 staticBinding 唯一的 predicate export。该 predicate 只能从已封存 target set 读取当前候选，以稳定 checkId 返回 assertion 和 finding。禁止动态发现、default/fallback 路由、固定 PASS、空 assertion、吞异常、直接写报告或调用其它 domain 私有 predicate。

54. Predicate 与逐 lawId 裁决

每个 staticBinding 精确返回：

```
{
  "lawId": "LAW-...",
  "bindingId": "<domain>.<law-key>.static",
  "checkId": "<domain>.<check-id>",
  "evidenceModes": ["static.ast"],
  "targetSet": {"queryRef": "scope:LAW-...#<targetSelectorId>:<64 lowercase
    hex>", "count": 1, "sha256": "<64 lowercase hex>"},
  "assertions": [
    {"assertionId": "<domain>.<assertion-id>", "passed": true,
      "evidenceRefs": ["ast:<repo-relative-path>#<selector>:<64 lowercase
      hex>"]}
  ],
  "findings": []
}
```

assertions 至少一项。passed=true 的 evidenceRefs 必须非空；纯 absence assertion 必须引用第 52 节 target set。passed=false、finding、异常、身份不一致或空证明均使 component FAIL。

READY 与 RELEASE 对每个 active lawId 产生一个最终裁决。CHANGE 不生成全量逐 lawId 裁决。

law 裁决只允许：

```
PASS：本等级要求的证据完整且无 finding；
FAIL：要求不成立、禁止条件出现或本等级 required evidence 失败。
```

每个 law 同时列出 declared、required 与 deferred evidenceModes。READY 只允许 dynamic.* 进入 deferred；RELEASE 的 deferred 必须为空。

55. evidenceRef

evidenceRef 精确使用 <kind>:<locator>#<selector>:<sha256>。kind 只允许 file/text/ast/json/yaml/sql/dependency/config/test/exam/dynamic/scope；locator 是仓库相对 path 或真实 locator；selector 必须稳定；sha256 是当前内容的 64 位小写十六进制 SHA-256。locator 或 selector 中的 # 与 : 必须 percent-encode。scope 必须引用 lawId 及 law scope hash；引用 staticBinding 时还必须引用 targetSelectorId 与 target-set hash。law scope hash 是 {lawId,appliesTo,excludes,ownedPaths,observedPaths,contractRefs} 第 30 节 canonical JSON bytes 的 SHA-256。

自然语言“见代码”“已经测试”“应该没问题”不是 evidenceRef。

56. 唯一逐法条质量报告

READY 与 RELEASE 每次只生成一份报告：

```
{
  "schemaVersion": 2,
  "level": "READY",
  "baseSha": "<git object id>",
  "candidateFingerprint": "<64 lowercase hex>",
  "candidateId": null,
  "registrySha256": "<64 lowercase hex>",
  "laws": [
    {
      "lawId": "LAW-...",
      "declaredEvidenceModes": ["static.ast", "dynamic.service"],
      "requiredEvidenceModes": ["static.ast"],
      "deferredEvidenceModes": ["dynamic.service"],
      "scopeEvidenceRefs": ["scope:LAW-...#<law-scope>:<64 lowercase hex>"],
      "components": [
        {
          "bindingId": "<domain>.<law-key>.static",
          "component": "static",
          "executorId": "<domain>.<check-id>",
          "evidenceModes": ["static.ast"],
          "outcome": "PASS",
          "targetSet": {"queryRef": "scope:LAW-...#<targetSelectorId>:<64
            lowercase hex>", "count": 1, "sha256": "<64 lowercase hex>"},
          "assertions": [
            {"assertionId": "<domain>.<assertion-id>", "passed": true,
              "evidenceRefs": ["ast:<repo-relative-path>#<selector>:<64
              lowercase hex>"]}
          ],
          "findings": []
        }
      ],
      "outcome": "PASS",
      "findings": []
    }
  ],
  "mutants": [
    {
      "mutantId": "<negativeCaseId-or-core-mutant-id>",
      "bindingId": "<bindingId-or-null>",
      "expectedCode": "COMPONENT_FAILED",
      "actualCode": "COMPONENT_FAILED",
      "evidenceRefs": ["test:<repo-relative-path>#<case>:<64 lowercase
        hex>"],
      "outcome": "PASS"
    }
  ],
  "outcome": "PASS"
}
```

报告顶层只允许 schemaVersion/level/baseSha/candidateFingerprint/candidateId/registrySha256/laws/mutants/outcome；law 对象只允许 lawId/declaredEvidenceModes/requiredEvidenceModes/deferredEvidenceModes/scopeEvidenceRefs/components/outcome/findings。

level 只允许 READY、RELEASE。READY 的 candidateFingerprint 必须非空且 candidateId 为 null；RELEASE 两者均必须非空，deferredEvidenceModes 必须为空。

laws[] 按 lawId 排序且精确覆盖全部 active lawId。declared 精确等于 law.evidenceModes，required 与 deferred 互斥且并集等于 declared。scopeEvidenceRefs 必须非空、引用当前 lawId 并绑定第 55 节 law scope hash；不得共享其它 lawId 的 scopeRef。components 与 required bindings 一一对应，并对 required evidenceModes 恰好覆盖一次。任一缺失、重复、额外项、空 assertion、空证据、component FAIL 或 finding 均使 law FAIL。顶层只在全部 law PASS 时 PASS。

component 对象只允许 bindingId/component/executorId/evidenceModes/outcome/targetSet/assertions/findings。component 只允许 static/exam/proof；executorId 必须精确等于 binding 的 checkId、examId 或 proofId。static 的 targetSet 必须精确等于第 52 节封存集合；exam、proof 的 targetSet 必须为 null。assertions 非空，每项只允许 assertionId/passed/evidenceRefs；任一 passed=false 必须使 component FAIL，任一 passed=true 必须有非空 evidenceRefs。

finding 对象只允许 code/evidenceRefs；code 必须是确定性阻断码，evidenceRefs 必须非空。PASS 对象的 findings 必须为空，FAIL 对象的 findings 必须非空。

mutants[] 每项只允许 mutantId/bindingId/expectedCode/actualCode/evidenceRefs/outcome，按 mutantId 排序且无重复。binding negativeCase 的 mutantId、bindingId 与 expectedCode 必须精确等于第 20 节声明；治理核心 mutant 的 bindingId 为 null。actualCode 与 expectedCode 精确相等、evidenceRefs 非空时 outcome 才能 PASS。READY 精确覆盖第 78 节 READY catalog；RELEASE 精确覆盖完整 catalog。缺失、重复、额外项或任一 mutant FAIL 均使顶层 FAIL。

顶层、law、component 与 mutant outcome 只允许 PASS、FAIL。不存在 NOT_APPLICABLE、SKIPPED_PASS、warning PASS 或 partial PASS。

报告不得再拆成平行 decision、component/law/Owner/CI receipt 或其它同义 verdict 层。

57. 报告写入

唯一 writer 只写第 56 节报告：

```
本地 READY、RELEASE：scripts/audit/reports/law-quality.report.json；
CI：仓库外 ephemeral path。
```

报告只证明对应 baseSha/candidateFingerprint/candidateId/registrySha256/level，不取得法案主权，不作为下一次证据输入。

58. 分级质量流程

```
CHANGE：scope -> 受影响 project checks 与普通测试 -> PASS | FAIL；
READY：source -> registry -> ownership -> 全部 projectChecks -> 全部 active
  Static -> 全部 active Exam -> required mutants -> 一份逐法条报告 -> PASS | FAIL；
RELEASE：READY -> candidate binding -> 全部 projectChecks -> 全部 active bindings
   -> 完整 mutant catalog -> candidate recheck -> 一份逐法条报告 -> proof package ->
  PASS | FAIL。
```

不得在各步骤之间生成可独立宣称整体通过的中间 PASS 文件。

59. 三类证据不得互相冒充

```
Static Audit 证明静态结构；
Exam 证明确定性行为；
Dynamic Proof 证明真实外部运行。
```

任何证据只能证明其实际观察到的对象。

60. Exam 的建立与适用

只有 active law 声明 exam.* evidenceMode 时才建立 Exam；law 未声明时不得追加。READY 与 RELEASE 执行全部 active examBindings。

Exam 代码可以共享，但必须在本次 quality run 中按 bindingId 执行并产生独立结果，不得复用历史结果。结果必须携带精确 examId、evidenceModes、非空 assertions 和 findings，并使用第 55 节 evidenceRefs 绑定输入、seed、超时、输出与当前候选。输入、seed、超时和失败用例必须可复放。

61. Dynamic Proof 的时点

只有 active law 声明 dynamic.* evidenceMode 时才建立 Dynamic Proof。RELEASE 执行全部 active proofBindings；CHANGE 与 READY 不执行，READY 必须如实列入 deferredEvidenceModes。

Dynamic Proof 只证明真实浏览器、服务、数据库、存储、provider、授权、恢复或性能事实。每个 proofBinding 必须在本次 RELEASE 的 candidateId 绑定后直接观察并产生独立结果，不得复用历史观察或旧 artifact 作为当前结果。结果必须携带精确 proofId、evidenceModes、非空 assertions 和 findings，并使用第 55 节 evidenceRefs 绑定真实 locator、观察值、候选与证据 artifact hash。凭证不得写入仓库或报告。

不可逆动作必须有人类逐字授权。cleanup 失败即 Dynamic Proof 失败。

外部动态条件暂时不可用时：

```
CHANGE 可以完成；
READY 可以按其静态和测试条件裁决；
RELEASE 必须失败或等待。
```

62. 证据适用性

appliesTo/excludes、Manifest paths 和 contractRefs 只定义检查的 target set，不得使 active law 在 READY 或 RELEASE 退出分母。

```
CHANGE：只执行与实际差异相交的 binding，不产生全局裁决；
READY：全部 active lawId 进入报告，全部 static.* 和 exam.* 为 required，全部 dynamic.* 为
  deferred；
RELEASE：全部 active lawId 的全部 declared evidenceModes 为 required，deferred 为空。
```

分母必须同时满足：

```
activeLawIds
= Manifest.lawBindings.keys
= registry.enforcement lawIds
= report.laws lawIds

required (lawId, bindingId, evidenceMode)
= PASS components 中的 (lawId, bindingId, evidenceMode)
```

上述等式均为集合精确相等；缺失、重复、额外项或临时降低 evidenceModes 均 FAIL。

63. 唯一质量入口

仓库只保留一个 canonical quality 入口 quality:check，并接受 CHANGE、READY、RELEASE 三级参数。编程语言和内部文件组织由仓库决定。

level 必须与第 35 节分类一致。

63.1 CHANGE

只检查允许路径、实际差异，以及与本次差异相关的 project checks 和普通测试。CHANGE 不执行全仓 law history、全量 registry/ownership、全量治理测试或逐 lawId 报告。PASS 只表示本次变更在 CHANGE 等级闭合。

63.2 READY

在 CHANGE 基础上执行 source、registry、ownership、全部 projectChecks、全部 active Static、全部 active Exam、required mutants 和逐 lawId 报告。PASS 只表示候选达到 READY 条件。

63.3 RELEASE

绑定完整 candidateId，重新执行全部 projectChecks、全部 active bindings 和完整 mutant catalog，生成第 64.2 节候选证明包。全部通过后才可产生 SYSTEM_PROVEN。

64. CI 与发布证明包

64.1 CI

CI 只能调用同一 canonical quality 入口。普通 PR 使用 CHANGE 或 READY；自动事件不得提升为 RELEASE。

CI 报告写入仓库外临时目录，不修改 source、registry、runtime 或仓库内 report cache。最终成功或失败只认质量进程退出码。

64.2 RELEASE proof package

只有 RELEASE 的全部检查和逐法条报告通过后才生成候选证明包；证明包成功写入并完成 candidate recheck 后，quality:check 才能以 RELEASE PASS 退出。证明包只包含：

```
candidateId；
Git commit；
freeze、两份 active law 与 registry hash；
逐法条质量报告 hash；
Dynamic Proof hash；
全部 required 跨仓合同版本与 participant release hash。
```

证明包不得包含新的规范义务或自然语言 READY。candidate 变化后，旧证明包只具有历史追溯意义。

第九部分 | 根治理规则与建制

65. blank bootstrap

空白仓首次建制只建立：

```
本说明书与 freeze；
AGENTS；
基础法案与功能法案；
第 66 节项目根治理与法定元治理规则；
按实际生产路径建立的 Manifests；
唯一 law registry；
一套普通治理测试；
READY 与 RELEASE 使用的一份逐法条质量报告；
quality:check。
```

blueprint、spec、迁移桥和跨仓合同只在第 22、24、43、69 节条件成立时建立；Exam 与 Dynamic Proof 只依 active law evidenceModes 建立。不得预建空壳。

bootstrap 完成 CHANGE 与 READY 后可以继续分步建设；只有进入 RELEASE 时才要求完整 candidate binding 和全部 RELEASE required evidence。

66. 项目根治理规则

基础法案保存长期稳定、跨版本持续适用的根治理规则。不得把无独立长期风险的 parser、compiler、report writer、测试夹具或某个实现步骤分别立为元法案。

本节八条 V2 项目根规则是最低必备根集合，不是《基础法案》的全部内容，也不限制《基础法案》的法条或 capabilityDomain 数量。它们共同使用：schemaVersion=2、status=active、tier=P0、capabilityDomain=project-governance，并各自以对应 V1 作为 supersedesLawId；enforcementMode 由第 14.3 节从本条证据推导。每条“要”正文进入 summary，“不要”中以分号分隔的句子依次进入 forbidden，“证据”进入 evidenceModes。

《基础法案》还必须保留以下 lawKey 的当前 active major：

```
LAW-LAW-SYSTEM-001、LAW-LAW-SYSTEM-002、LAW-LAW-SYSTEM-003、LAW-LAW-SYSTEM-004、
  LAW-LAW-SYSTEM-005、LAW-LAW-SYSTEM-006、LAW-LAW-SYSTEM-007、LAW-LAW-SYSTEM-00
  8；
LAW-AUDIT-SYSTEM-001、LAW-AUDIT-SYSTEM-002、LAW-AUDIT-SYSTEM-003、LAW-AUDIT-SYS
  TEM-004、LAW-AUDIT-SYSTEM-005、LAW-AUDIT-SYSTEM-006、LAW-AUDIT-SYSTEM-007、LAW
  -AUDIT-SYSTEM-008、LAW-AUDIT-SYSTEM-009、LAW-AUDIT-SYSTEM-010、LAW-AUDIT-SYST
  EM-011、LAW-AUDIT-SYSTEM-012、LAW-AUDIT-SYSTEM-013、LAW-AUDIT-SYSTEM-014。
```

上述 lawKey 的最低不变量精确为：

lawKey	必须独立执法的风险
LAW-LAW-SYSTEM-001	两份 active law source，零 shadow law
LAW-LAW-SYSTEM-002	法条原子语法与标题镜像一致
LAW-LAW-SYSTEM-003	lawId、domain 与历史谱系唯一
LAW-LAW-SYSTEM-004	active lawId 与原子 binding 全分母相等
LAW-LAW-SYSTEM-005	blueprint/spec 机器合同唯一
LAW-LAW-SYSTEM-006	唯一 compiler 与唯一 registry projection
LAW-LAW-SYSTEM-007	registry exact-byte 与只读复验
LAW-LAW-SYSTEM-008	active law 与本说明书修订只经明确人类授权
LAW-AUDIT-SYSTEM-001	Audit 的执法计划只来自唯一 registry projection
LAW-AUDIT-SYSTEM-002	每条 active law 的 required evidence binding 完整
LAW-AUDIT-SYSTEM-003	一个候选的证据索引只存在于唯一逐法报告
LAW-AUDIT-SYSTEM-004	每个含 static.* 的 lawId 具有独立 predicate 与结果
LAW-AUDIT-SYSTEM-005	任一 finding 强制 FAIL

lawKey	必须独立执法的风险
LAW-AUDIT-SYSTEM-006	Static 不执行或消费 Exam、Dynamic
LAW-AUDIT-SYSTEM-007	测试与 fixture 不得作为业务静态证据
LAW-AUDIT-SYSTEM-008	陈旧 registry projection 必须 FAIL
LAW-AUDIT-SYSTEM-009	candidate drift 使本次全部证据失效
LAW-AUDIT-SYSTEM-010	禁止固定 PASS 与空证明
LAW-AUDIT-SYSTEM-011	law 所属 domain、Manifest、binding、executor 结果与 report lawId 的身份一致
LAW-AUDIT-SYSTEM-012	历史方案与旧报告对当前 Audit 零输入
LAW-AUDIT-SYSTEM-013	复合证据任一 required component 缺失即整法 FAIL
LAW-AUDIT-SYSTEM-014	唯一最终汇总与唯一报告 writer

每个 lawKey 必须保持表中独立风险、第 14 节原子语法与本章执法强度，不得由 project-governance 汇总条款替代、弱化或在迁移中省略。

66.1 [LAW-PROJECT-GOVERNANCE-001-V2] 权威顺序与规范主权

要：必须把人类任务授权只用于确定本次范围，并按照说明书、基础法案、功能法案、施工图、spec、Manifest、registry、任务记录、实现和当前证据的顺序裁决冲突。

不要：不得让任务提示词、代码、测试、generated artifact、历史报告、技术便利或局部实现覆盖上级规范主权；不得在未获人类明确授权时修改 active law，也不得让 evidence 反向取得规范主权。

证据：static.text、static.contract、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.2 [LAW-PROJECT-GOVERNANCE-002-V2] 唯一真相、唯一 Owner 与无主阻断

要：必须使每项生产事实、能力、合同、入口、registry、compiler、runner、report writer、生产路径和外部 surface 具有一个现役真相源及一个 primary Owner。

不要：不得建立未按第 43.2 节登记的 compatibility、fallback、shadow、bridge 或 dual-write，也不得建立长期平行主链、第二 registry、第二 Owner map、第二 compiler 或第二质量 verdict；不得让 projection、缓存、历史副本、observedPaths、shared/common 容器或消费关系取得源头主权；不得先创建无 Owner 的文件、目录、入口、依赖、配置、合同或外部 surface 再补治理。

证据：static.filesystem、static.dependency、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.3 [LAW-PROJECT-GOVERNANCE-003-V2] Codex 任务范围不可漂移

要：必须使 READ_ONLY 直接读取且零落盘，并使 CHANGE 在首次目标写入前记录真实 baseSha、允许写删路径及完整初始闭包，运行只读 entry，结束时核对 dirty target 与实际差异。

不要：不得顺手深化、扩域、重构、预留未来能力、修改允许范围之外的路径或先写后补；不得覆盖用户原有改动、篡改 baseline/change record 逃避检查或为只读任务建立 runtime 事务。

证据：static.filesystem、static.contract、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.4 [LAW-PROJECT-GOVERNANCE-004-V2] 完整终态与交付等级真实

要：必须使 active 法案、施工图和规格直接描述完整生产终态，并对 CHANGE、READY、RELEASE 分级裁决且明确每一级已完成与未完成内容。

不要：不得把阶段、MVP、preview、未来接入、默认关闭、人工晋级、待完善、临时壳、静态检查或部分证据冒充完整终态或生产发布完成；不得用任务拆分改变已经授权能力的完整交付分母。

证据：static.text、static.contract、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.5 [LAW-PROJECT-GOVERNANCE-005-V2] 有期限迁移与旧链物理退出

要：必须使迁移桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic 或 adapter 具有 Owner、监控、期限、删除日期、回滚条件和物理删除验收。

不要：不得形成两个长期真相源、复活旧入口、旧 runner、旧 generated 或旧 Owner mapping；不得到期不删，也不得把迁移状态冒充最终终态。

证据：static.contract、static.config、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.6 [LAW-PROJECT-GOVERNANCE-006-V2] 证据失败关闭与禁止伪 PASS

要：必须使每个 active lawId 绑定全部必需证据组件，使 Static、Exam、Dynamic 只证明各自对象，并使逐法条报告绑定当前等级和候选。

不要：不得用 warning、partial、coverage、人工声明、历史结果、fixture、mock、错误证据类型或单一组件 PASS 放行整条法案；不得让 Static、Exam 或 Dynamic Proof 互相冒充，也不得在证据缺失、失败、过期或 candidate 漂移后继续放行。

证据：static.contract、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.7 [LAW-PROJECT-GOVERNANCE-007-V2] 源头冻结与历史零输入

要：必须以 freeze 仅绑定说明书及已启用派生物，以当前 source checks 验证 AGENTS、active law 与 registry，并使历史材料对当前执行分母零输入。

不要：不得由普通任务自动改写说明书、AGENTS、freeze、active law 或工具链 pin；不得让历史文件、旧报告、retired law、旧 generated 或旧入口取得当前主权或成为当前质量输入。

证据：static.text、static.config、static.json、exam.property。

违法后果：使用第 16 节 P0 固定文本。

66.8 [LAW-PROJECT-GOVERNANCE-008-V2] 任务闭合、候选绑定与单一终局报告

要：必须使任务完整闭合，使唯一逐法报告绑定 RELEASE candidateId 与全部 required evidence，并使唯一 proof package 绑定 Git commit、法案、registry、报告、Dynamic Proof 及 required 跨仓结果。

不要：不得遗漏实现、合同、错误恢复、证据、generated projection、旧路径退休或跨仓参与者闭合；不得以 CHANGE、READY、旧候选证据、局部 Owner PASS 或缺失外部证明的状态宣称 SYSTEM_PROVEN。

证据：static.contract、exam.property。

违法后果：使用第 16 节 P0 固定文本。

67. 禁止递归元法案工厂

治理系统不得固定要求：

```
第 66 节之外缺少独立长期风险的元法案；
每条法案一份 spec 文件；
每条法案一份 blueprint 文件；
每条法案一份 predicate 文件；
每条法案一份 Exam 文件；
每个组件一份 mutant 文件；
```

```
多份内容重叠的 generated projection；
多层 decision、receipt 和 report verdict。
```

禁止的是机械复制文件，不是第 20、51 至 56 节要求的一法一逻辑执法身份。新增治理 law 只有在第 66 节不能覆盖一个长期独立风险，且该风险可以写成可裁决规则时才合法。

业务 law 按 capabilityDomain 写入功能法案；其 blueprint、spec、Exam、Dynamic Proof 和专项测试均按实际拓扑、合同及 evidenceModes 建立。

68. 唯一命令面

跨实现稳定的公共治理命令身份只允许固定为：

```
governance:entry
governance:scope:check
audit:build
audit:projection:check
quality:check
```

它们分别承担入口检查、范围检查、registry 构建、registry 复验和唯一质量裁决。普通治理测试由 quality:check 调用，不冻结独立命令名。编程语言、内部脚本文件名、模块拆分、依赖和测试框架均由仓库决定，不属于本说明书冻结内容。

Exam 或 Dynamic Proof 存在时可以拥有独立执行命令，但只能把结果交给 canonical quality 汇总，不得形成第二质量 verdict。

第十部分 | 既有仓库迁移与新增能力

69. existing migration

既有仓库允许分步迁移。存在迁移桥时，必须在第 24 节 specKind=migration 的 spec 的 contract 中记录：

```
{
  "currentAuthority": "<identity>",
  "targetAuthority": "<identity>",
  "compatibilityMechanisms": [],
  "startDate": "YYYY-MM-DD",
  "monitoring": [],
  "rollbackCondition": "...",
  "maximumDurationDays": 30,
  "removalDate": "YYYY-MM-DD",
  "removalAcceptance": []
}
```

没有迁移桥的普通分步建设不创建 migration spec，不建立第二 runtime 任务记录。

maximumDurationDays 必须是 migration contract 按实际迁移声明的正整数；上例的 30 只是示例，不是全系统固定上限。removalDate 不得晚于 startDate 加 maximumDurationDays。

规则固定：

1. 中间步骤只能声明实际完成的 CHANGE 或 READY；最终步骤通过全部发布条件后才可声明 RELEASE。
2. 旧、新治理并存时必须指定当前裁决主权，不得双主权。
3. 临时 bridge、adapter 或 dual-read/write 依第 43.2 节登记和删除。
4. 完成最终 RELEASE 前，旧 normative source、旧 registry、旧 quality verdict 和过期 bridge 必须清零。
5. 治理迁移不得顺带修改未授权业务功能或工具链。

70. 新增 capabilityDomain

新增 domain 需要人类明确授权，并在首次生产写入前确定：

```
domain 名称；
primary Owner；
active lawIds；
ownedPaths；
与相邻 domain 的合同边界；
本阶段交付等级。
```

施工图、spec 依实际拓扑和合同建立，Exam、Dynamic Proof 只依 active law 的 evidenceModes 建立。不得因为尚未进入 RELEASE 而伪造空文件，也不得把未完成项写成已完成。

71. 新增 lawId

新增 lawId 必须同时完成：

```
唯一 law-record；
合法 lawId、domain 和当前版本关系；
Manifest.lawIds；
Manifest.lawBindings 原子 binding；
对应 inspector/Exam/Proof declaration 与每个 binding 的 negativeCase；
唯一 registry 重建；
进入 READY 或 RELEASE 时的逐 lawId 质量报告。
```

只有法条实际涉及机器合同才增加 spec；只有其 evidenceModes 声明对应类型时才增加 Exam 或 Dynamic Proof。不得机械创建一法一文件，也不得省略一法一逻辑执法身份。

72. lawId 修订和退役

72.1 修订

summary、forbidden、appliesTo/excludes、Owner、evidenceModes、tier 或依赖义务发生规范变化时，创建下一 major version并记录 supersedesLawId。同一 law-amendment 必须删除旧 active block，同步替换 Manifest binding 及其 executor/negativeCase declarations，并重建 registry；新、旧 major 不得同时 active。

法条 major 与 spec version 独立：只改变业务合同而未改变法条义务时，只递增 spec version，不触发本说明书或无关 lawId 修订。破坏性合同变化可以在声明的兼容窗口内保留旧、新 spec version；必须满足第 43.2 节，并在期限内删除旧版本。兼容窗口不改变唯一规范主权：新版本生效范围和旧版本退休范围必须明确。

72.2 退役

退役必须删除 active block 和 Manifest 当前引用，移除独占入口，并通过唯一 builder 重建 registry 以移除当前记录；不得手改 generated registry。历史只保留在 Git 或明确 HISTORY_ONLY 路径。

被其它 active law、合同或消费者继续引用时不得宣称退役完成。

第十一部分 | 必须阻断的漂移

73. 架构漂移

必须阻断：

未经授权新增第二长期入口、第二 registry、第二 Owner map、第二 compiler 或第二质量 verdict；
把业务主权下沉到 shared/common/utils；
让 generated registry、Manifest、runner 或报告反向取得法案主权；
用临时迁移机制长期承载正式主链。

74. 范围漂移

必须阻断：

修改 change record 之外的路径；
为格式统一修改无关法案或 domain；
为未来支持增加未授权接口；
吞异常、缩小证据分母或覆盖用户原有改动；
发现旧代码后未经授权全面清理。

75. 证据漂移

必须阻断：

文件存在即 PASS；
token、数量或 Owner 存在即 PASS；
fixture 成功即生产成功；
历史报告即当前证据；
Static 即真实外部运行；
截图即完整动态证明；
CHANGE 或 READY 即 RELEASE。

76. 迁移漂移

迁移桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic、adapter 和兼容窗口本身不是违法。

必须阻断：

无 Owner；
无监控；
无回滚条件；
无最长期限或删除日期；
旧、新主权不明确；
到期未删除；
把迁移状态冒充最终 RELEASE；
以兼容为名永久保留两个真相源。

77. 文档漂移

必须阻断：

Codex 把自己的理解写成新法案；
在法条中加入背景故事、方案比较和开发日志；
在施工图或 spec 中复制法案；
在 AGENTS 或 README 维护第二法案清单；
在历史报告声明当前 READY、RELEASE 或 SYSTEM_PROVEN。

第十二部分 | 测试与独立自检

78. 普通测试与专项 mutation

mutation catalog 必须覆盖 parser、registry builder、scope checker、quality aggregator、禁止伪 PASS 的关键分支，并为每个 binding 登记至少一个第 20 节 negativeCase。

治理核心 mutant 的最低分母是：第 21、34、41 节每个确定性阻断码至少一个隔离用例，以及固定 PASS、空 assertions、空 evidenceRefs、错误 evidenceMode、finding 未使失败、分母缺失、重复或额外项各一个隔离用例。实现另行声明的 core mutant 立即进入完整 catalog，不得在 RELEASE 排除。

mutation 规则：

```
一个 mutant 只改变一个关键条件；
每个 binding negativeCase 必须使该 law 的 summary 不成立或使一项 forbidden 成立；仅删除结果、篡改
  outcome 或破坏 report wrapper 不构成 binding negativeCase；
必须触发预期失败；
在隔离目录运行；
不得改变原工作树；
负向用例可以共享构造器，但必须按 bindingId 产生独立的预期失败；
负向 Proof 只破坏隔离的证据输入或裁决条件，不得对真实外部系统执行破坏动作；
不得以 mutant 数量或覆盖率作为 PASS。
```

CHANGE 只执行本次差异影响的 mutant。READY 执行全部 active staticBinding、examBinding negative cases 和完整治理核心 mutants。RELEASE 执行完整 catalog，包含 proofBinding negative cases。未触发精确 expectedCode 即 MUTANT_NOT_DETECTED。

79. 独立自检

说明书定稿、治理主链变化或大规模迁移后，独立审查至少检查：

```
权威是否唯一；
任务范围是否越界；
registry 是否忠实投影 active laws；
逐法 binding、required evidence 和报告分母是否精确相等；
逐法条报告是否使用正确证据；
每个 PASS assertion 是否有非空可复放证据；
负向用例是否能使对应 binding 失败；
CHANGE、READY、RELEASE 是否被如实区分；
迁移机制是否具有期限并可删除；
是否存在固定 PASS、遗漏分母或第二 verdict。
```

审查输出 finding，不以总体评价或覆盖率代替具体证据。

第十三部分 | 最终验收与冻结完成定义

80. 法案正文验收

active 法案容器精确两份；
基础法案完整包含第 66 节项目根规则、law-system 与 audit-system 法定 lawKeys；
功能法案包含实际业务与随能力持续迭代的治理 law；
全部 lawId、domain、supersedes 和当前版本关系成立；
forbidden 非空；
active law source 精确为固定两份；
不得存在只为递归证明治理实现自身而设立的元法案。

81. registry、施工图与规格验收

唯一 generated artifact 是 law-registry.generated.json；
registry 忠实投影全部 active laws、Owner、contract refs 和 enforcement bindings；
active law、Manifest binding、registry 和报告分母精确相等；
registry builder deterministic，projection check 只读；
施工图和 spec 只在实际复杂度要求时存在；
不存在一法一 spec 文件、一法一 blueprint 文件或空模板目录；
跨仓合同具有明确 provider、version 和 consumer pin。

82. Owner 与 scope 验收

每个生产路径和合同一个 primary Owner；
READ_ONLY 零 runtime 写入；
CHANGE record 只含 baseSha、allowedWritePaths、allowedDeletePaths；
实际差异未超出允许路径；
未获授权的 dirty target 未被触碰；
新增 surface 具有现役依据。

83. 交付与迁移验收

CHANGE、READY、RELEASE 分级明确；
阶段、MVP 和 preview 未冒充生产完成；
迁移桥具有 Owner、监控、回滚、期限、删除日期和删除验收；
跨仓通过版本化合同独立部署；
只有整体 RELEASE 要求必要参与仓发布结果；
外部 Dynamic Proof 不可用时只阻断 RELEASE，不阻断合规 CHANGE。

84. 质量验收

一套普通治理测试；
一份逐 lawId 质量报告；
每个 active lawId 有且只有一个原子 binding record；
每个含 `static.*` 的 law 有唯一 checkId 与 target selector；
每个 binding 有非空可执行 negativeCases，并触发精确 expectedCode；
每个 required evidenceMode 恰好由一个 PASS component 覆盖；
每个 PASS assertion 有非空 exact evidenceRefs；
READY 仅 dynamic.* 可 deferred，RELEASE 的 deferredEvidenceModes 为空；
Static、Exam、Dynamic 不互相冒充；
READY 执行全部 active Static 和 Exam，RELEASE 执行全部 active bindings；
第 78 节全部 required mutants 触发预期失败；
不存在平行 decision、receipt、report 或多层 verdict；
任一 finding、错误证据类型或 candidate drift 必须 FAIL；
CI 调用同一 canonical quality 入口；
RELEASE 才生成候选证明包。

85. 状态与完成定义

SPEC_FROZEN：说明书和启用的派生物由当前有效 freeze manifest 保护；
CHANGE_PASS：本次允许差异通过与差异相交的检查；
READY：全部 projectChecks 通过，全部 active lawId 进入报告，全部 Static、Exam 与 READY required mutants 通过，仅 `dynamic.*` deferred；
SYSTEM_PROVEN：当前 RELEASE candidate 的全部 projectChecks、active bindings、完整 mutant catalog 与 required 跨仓结果通过。

附录 A | AGENTS.md 固定正文

bootstrap 生成的 AGENTS 规范性正文精确为：

```
# Codex Governance Entry

本仓库使用冻结的法案系统。

1. READ_ONLY 直接读取并回答，不落盘、不加锁、不建立 runtime 任务事务。
2. CHANGE 首次目标写入前只建立三字段 change record，并运行只读 `governance:entry`。
3. 修改不得超出 change record；扩大范围必须先更新同一 record。
4. READY 不等于 RELEASE；不得以静态检查、普通测试或部分证据冒充生产发布。
5. RELEASE 才绑定完整 candidateId，并执行全部 active law 声明的 Dynamic Proof。
6. 只允许一份 active law corpus、一份 law registry；每个生产路径和合同只允许一个 primary
   Owner；只允许一个 quality verdict。
7. 迁移桥、用于迁移的 feature flag、dual-read/write 或 shadow traffic 必须有
   Owner、监控、期限、删除日期和删除验收。
8. active law 只在明确人类授权下修改。
9. 任一 finding、候选漂移、错误证据类型或伪 PASS 必须失败关闭。
```

AGENTS.md 中上述正文由本说明书授权，但不属于 active law source，也不进入 SPEC_FROZEN 字节绑定；其符合性由治理入口检查。

附录 B | 最小机器合同

B.1 change record

精确使用第 36 节；本附录不维护第二份 schema。

B.2 Manifest

精确使用第 20 节；本附录不维护缩略 schema。

B.3 按需 blueprint machine block

精确使用第 22 节；本附录不维护第二份 machine block。

B.4 按需 spec

公共外壳精确使用第 24 节；B.7、B.8 分别定义两类跨仓 contract payload。

B.5 law registry

精确使用第 27、28 节；本附录不维护第二份 registry schema。

B.6 逐法条质量报告

精确使用第 56、57 节；本附录不维护第二份报告 schema。

B.7 跨仓 provider contract payload

本节不是独立 artifact；它是第 B.4 节 spec 在 specKind=cross-repo-contract 时的 contract 内容：

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo",
  "providerDomain": "provider-domain",
  "authorityArtifact": {
    "path": "repo/relative/path",
    "selector": "stable-selector",
    "sha256": "<64 lowercase hex>"
  },
  "consumerRepos": []
}
```

B.8 跨仓 consumer pin payload

本节不是独立 artifact；它是第 B.4 节 spec 在 specKind=contract-pin 时的 contract 内容：

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo",
  "providerDomain": "provider-domain",
  "providerSpecRef": "docs/specs/<provider-domain>/<spec-id>.spec.json",
  "providerCommitSha": "<git object id>",
  "contractSha256": "<64 lowercase hex>"
}
```

consumerRepos[] item 必须是 canonical owner/repo 字符串，按 UTF-8 bytes 排序且无重复；B.7、B.8 对象均拒绝未知字段。provider 与 consumer 可以独立部署；合同内容变化递增 version，破坏性改变同时声明兼容窗口和迁移期限。

contractSha256 绑定 provider spec 的 contract canonical JSON bytes；consumer pin 的 contractId、version、providerRepo、providerDomain 和该 hash 必须与 provider spec 完全一致。providerCommitSha 绑定包含该 provider spec 的提交。

B.9 RELEASE proof package

```
{
  "schemaVersion": 1,
  "candidateId": "<64 lowercase hex>",
  "gitCommit": "<git object id>",
  "freezeSha256": "<64 lowercase hex>",
  "baseLawSha256": "<64 lowercase hex>",
  "functionalLawSha256": "<64 lowercase hex>",
  "registrySha256": "<64 lowercase hex>",
  "lawQualityReportSha256": "<64 lowercase hex>",
  "dynamicProofHashes": [],
  "crossRepoParticipants": [
    {
      "contractId": "provider-domain.contract",
      "contractVersion": 1,
      "repo": "owner/repo",
      "releaseSha256": "<64 lowercase hex>"
    }
  ]
}
```

proof package 只在 RELEASE 的全部检查和逐法报告 PASS 后生成；其写入与 candidate recheck 通过后才能宣告 RELEASE PASS。不得用于 CHANGE 或 READY。

proof package 顶层及 participant item 均拒绝未知字段；crossRepoParticipants[] 按 contractId、contractVersion、repo 排序且无重复，并按第 30 节生成 canonical JSON。参与仓依赖必须无环；每个 package 只引用已经完成 RELEASE 的直接依赖仓 package，不得形成互相 hash 的递归绑定。

hash 身份固定为：

```
freezeSha256：当前有效 docs/governance/GOVERNANCE-FREEZE.json 原始字节的 SHA-256；
baseLawSha256、functionalLawSha256：对应 active law 文件原始字节的 SHA-256；
registrySha256：唯一 registry canonical JSON bytes 的 SHA-256；
lawQualityReportSha256：本次 RELEASE 逐法条质量报告 canonical JSON bytes 的 SHA-256；
dynamicProofHashes[]：逐法条质量报告实际引用的 Dynamic Proof artifact 的 SHA-256，去重后按小写
  hash 升序排列；
crossRepoParticipants[].releaseSha256：对应参与仓 RELEASE proof package canonical
  JSON 原始字节的 SHA-256。
```

不得为解释这些 hash 另建 receipt。